

0. 作战判断

桌面端 Agent 的胜负不在“会聊天”，而在 Harness 是否把计划、上下文、工具、失败、人工接管全部结构化，为可验证任务结果，并把高质量轨迹反哺模型训练。

Model + Harness = Agent 放到桌面端，模型只负责给出候选动作，Harness 决定五件事：任务是否被正确拆解、工具是否被正确调用、失败后是否能恢复、用户能否在关键节点接管、失败轨迹能否进入训练和回归。

- 成功不看模型自评，看状态差是否达成、人工是否验收、轨迹能否复用。
- 失败不当客服反馈看，拆成工具错误、上下文丢失、权限越界、恢复失败、验证失败五类。
- 桌面端不做聊天框，做任务控制台：计划、证据、权限、记忆、失败恢复全部显式化。
- 首月不追全能，只拿边界清晰、状态差可验证、失败轨迹高价值的任务场景。

JD 明示要求	系统短板	交付抓手
定义 Agent 是否真的帮助到人	DAU 和聊天轮次穿不透任务结果	ETCR 漏斗、分层指标、置信区间
与模型训练团队共同进化	产品侧失败轨迹不能直接进入训练	Negative Trajectory Pack、Tool Failure Taxonomy、回归集
理解 MCP / KV Cache / Memory	工具边界、上下文成本、长期记忆污染没有产品化	能力卡、预算账本、分级 Memory、权限域
内部任务反馈闭环	Dogfooding 容易论为主观吐槽	Harness Probe、任务轨迹 JSONL、接管包、周报面板

- AgentHelpJob / LeadPulse 的价值不在项目名，在于已经跑过 解析 → 匹配 → 生成 → 人工确认 → 追踪 的长链路拆解。
- 百度 AI 黑客松 4 小时全栈原型的价值不在速度，在于首周就能拉起轻量探针、埋点和实验控件。
- 179 篇内容、48.4 万曝光、3.72 万互动的价值不在流量，在于已经验证两类信号：非技术用户要可复制路径，开发者要工具分工、状态透传和接管边界。

1. 核心指标重定义：ETCR 取代 DAU 和聊天轮次

DAU 只能说明用户打开了产品。聊天轮次只能说明模型消耗了上下文。桌面端 Agent 必须按任务结果和状态差衡量。

1.1 ETCR

ETCR = 有效完成任务数 / 已接受任务启动数

有效完成同时满足：- 目标被结构化：任务意图、输入资产、边界、期望状态差被记录。- 计划被采纳：用户确认或默认授权的计划版本存在，后续执行没有越权。- 工具链可验证：关键工具调用有输入、输出、错误栈、重试路径、权限事件。- 结果被验收：自动 verifier、人工确认或 T+24h 未回滚规则至少命中一项。- 轨迹可复用：能进入 eval、负样本池或 SFT/RLHF 候选池。

AcceptedDone = StateDiffPassed + HumanAccepted + NoCriticalRollback + TraceReusable

不要把“模型说完成了”计入 AcceptedDone。

1.2 漏斗拆解

漏斗层	关键事件	主指标	失败信号
Intent Capture	输入被解析为任务对象	意图结构化率 ISR	澄清轮次过高、边界缺失
Plan Adoption	计划被确认或授权	计划采纳率 PAR	用户重写步骤、拒绝高风险工具
Tool Execution	工具按计划执行	工具成功率 TSR	schema mismatch、timeout、permission denied
Recovery	失败后恢复	失败恢复率 TFR	同错重复、无降级、无恢复包
Verification	结果验收	状态差通过率 SDR	测试失败、diff 偏离、人工回滚
Trace Reuse	轨迹进入训练 / 评估	轨迹可用率 TRR	上下文缺失、PII 无法脱敏、标签不一致

1.3 分层指标

用户层	不追求	真正目标	监控指标
非技术用户	多聊几轮	用模板完成第一个真实任务	TTFET、模板采纳率、人工确认通过率、撤销率、二次任务启动率
专业开发者	让 Agent 写更多代码	在边界内交付可审查 diff	计划采纳率、diff 接受率、测试通过率、人工接管率、越权阻断率

1.4 首批灰度实验

实验	A / B	成功判据	风险控制
计划确认机制	A 直接执行 / B 先展示计划、工具、影响文件、回滚点	B 组 ETCR 上升且人工接管率下降	高风险工具强制确认，只在低风险任务开放 A
显性剪枝机制	A 自动压缩 / B 展示保留与丢弃清单并允许 pin	B 组上下文修正率下降、重复解释次数下降	压缩前记录 context_manifest checksum
失败恢复机制	A 静默重试 / B 展示错误栈、备选路径、降级方案	B 组重复同错率下降，任务中断率不上升	超过重试阈值即阻断，保留人工接管 resume bundle

统计处理： - 早期样本小，不用均值拍板。 - 对 ETCR、PAR、TFR 用 Bootstrap 和 Wilson 区间。 - 按任务类型、用户层级、复杂度分层，避免低风险任务稀释真实问题。

2. Harness 与模型协同进化：从失败轨迹到训练信号

研究团队不需要“用户觉得不好用”，需要可压缩、可标注、可复现、可评估的轨迹。Harness 的职责是把产品侧失败转成训练和回归都能消费的数据。

2.1 Trajectory Event Schema

字段组	关键字段	用途
任务身份	task_id、session_id、 user_segment、task_type、 risk_level	按任务而不是聊天轮次聚合
输入与计划	user_goal_hash、 parsed_goal、plan_version、 planned_tools、consent_state	判断任务拆解和计划采纳是否正确
上下文	context_manifest、 memory_reads、file_reads、 MCP_resource_refs、 compaction_event	定位遗忘、污染、过载、压缩损失
工具链	tool_call_id、tool_name、 mcp_server_id、 args_schema_hash、 output_hash、error_stack	定位 Tool Use 失败和 schema 漂移
权限与接管	permission_prompt、 user_decision、 takeover_timestamp、 takeover_reason	形成偏好样本和安全边界样本
验证	verifier_type、state_diff、 test_result、 rollback_event、 final_acceptance	把完成从主观描述变成状态差

2.2 Negative Trajectory Taxonomy

- 工具选择错误：调用了能返回信息但不能改变状态的工具，或在可用 MCP 工具中选了低置信路径。
- 工具参数错误：schema 字段遗漏、类型错误、路径错误、权限作用域错误。
- 上下文丢失：压缩后丢失路径级规则、局部 CLAUDE.md、用户限制或项目约定。
- 权限边界错误：原计划只读，执行阶段升级为写操作、外部请求或删除。
- 恢复错误：同一错误重复重试、无替代工具、无降级计划、无人工接管包。
- 验证错误：工具执行成功但状态差未达成；代码有 diff 但测试没跑；文档生成但没有来源证据。

2.3 训练回流机制

数据包	内容	给训练 / 研究团队的用途
SFT Candidate	成功轨迹：目标、计划、最小上下文、正确工具调用、验证证据	训练任务模板、工具参数生成、计划表达
Preference Pair	同一任务的计划 A/B、工具路径 A/B、接管前后输出	训练计划偏好、风险边界、何时询问用户
Negative Data	失败调用、错误栈、上下文快照、人工修正动作	训练拒绝错误路径、失败恢复、 schema 修正
Eval Regression Set	高频失败任务 + 标准 verifier + 期望状态差	每次模型 / Harness 变更前后跑回归，防止倒退

最小包结构： {task_type, user_goal, allowed_tools, context_manifest, failed_action, error_stack, human_fix, expected_next_action, verifier, final_state_diff}

2.4 数据质量门槛

- 可复现：从 context_manifest 和工具输入能重放到同一错误。
- 可脱敏：原文、文件路径、 token 、账号信息先过 PII scrubber 。
- 可标注：每条失败只允许一个主因标签，副因进入辅助标签。
- 可评估：必须绑定自动 verifier 或人工验收依据。
- 可分层：按用户层、任务类型、工具类型、风险级别切分。

3. 竞品逆向：Claude Code / Codex / OpenClaw 的桌面端短板

竞品不是用来照抄。竞品文档暴露的是桌面 Agent 的约束：上下文不是无限的，工具权限不是模型能保证的，云端异步不是桌面实时控制台的替代品，本地优先也不等于自动安全。

3.1 Claude Code：上下文强，显式治理不足

观察点	公开机制	致命弱点	DeepSeek 抓手
长上下文维护	会话里混有指令、文件、模型回复、隐藏内容； /compact 用结构化摘要替换历史	压缩后 path-scoped rules 、 nested CLAUDE.md 可能丢失，直到相关文件再次读取	Context Ledger：来源、寿命、优先级、触发条件、剪枝理由、压缩前后差异报告
权限边界	权限由 Harness 强制，存在 default 、 plan 、 auto 、 dontAsk 、 bypassPermissions 等模式	长任务会诱导用户牺牲边界换效率	风险分层授权：读、写、外网、删除、凭据访问分别授权，并写入计划 diff

观察点	公开机制	致命弱点	DeepSeek 抓手
失败恢复	遇到 <code>auto-compaction thrashing</code> 时建议切块、手动压缩、 <code>subagent</code> 、 <code>clear</code>	恢复策略偏手动，用户要知道怎么切块和清理上下文	自动生成 <code>Recovery Bundle</code> ：超大输出摘要、建议切片范围、可迁移 <code>subagent prompt</code> 、恢复前后状态差

3.2 OpenAI Codex：云端隔离强，桌面实时纠偏弱

观察点	公开机制	致命弱点	DeepSeek 抓手
任务形态	Codex web 把任务放到云端后台并行执行	异步适合清晰代码任务，不适合多应用、持续人工确认的桌面任务	默认做“可中断执行”：关键状态差前生成检查点，允许接管或回滚
安全隔离	云端任务在隔离环境中执行，本地 CLI 可读改跑代码	云端沙箱弱化了本机真实环境细节；桌面应用状态难完全复现	本地 Harness 记录环境指纹；低风险步骤本地执行，高成本步骤云端执行
协作模式	运行中 <code>course-correct</code> 能力弱于交互式编辑	用户只能等结果再修正，失败轨迹滞后	把“中途纠偏”做成默认控件：计划变更、工具失败、测试失败即时停靠在任务控制台

3.3 OpenClaw：本地优先强，安全边界必须产品化

观察点	公开机制	致命弱点	DeepSeek 抓手
入口形态	多消息渠道、 <code>gateway</code> 控制面、 <code>multi-agent routing</code> 、 <code>tools</code> 、 <code>skills</code>	入口越多，提示注入、身份伪造、跨渠道状态污染越难治理	先做可信入口、身份绑定、来源风险评分，不追入口数量
本地能力	<code>main session</code> 工具默认跑在 <code>host</code>	本地能力强，误操作半径也最大	所有 <code>state-changing action</code> 绑定 <code>consent_state</code> 、 <code>rollback_plan</code> 、 <code>state_diff verifier</code>
沙箱策略	<code>non-main session</code> 可进 <code>sandbox</code>	用户难判断当前会话处于哪个权限域	权限域常驻：当前代理、工具、目录、网络、凭据全部可见

4. 底层机制产品化：MCP、KV Cache、Memory 的桌面映射

`MCP`、`KV Cache`、`Memory` 不是名词考点，是桌面端 Agent 的产品约束。产品层要把它们变成用户可感知、系统可执行、数据可回流的机制。

4.1 MCP：从“能接工具”到“可治理工具生态”

产品对象	机制设计	验收指标
MCP Capability Card	显示工具列表、权限、数据范围、状态 改变风险、失败率、最近错误	工具误调用率下降，授权撤销率下降
Scope-based Consent	按目录、账号、项目、数据源、动作类型授权，不给全局“允许全部”	高风险工具阻断率上升且 ETCR 不下降
Tool Contract Test	接入前跑 schema validation、dry-run、mock output、错误码映射	schema mismatch 率下降，失败可回放率上升
MCP Health Router	对同类工具维护健康度、延迟、权限可用性，失败时切换备选路径	TFR 上升，重复同错率下降

4.2 KV Cache 压力：产品层不能假装上下文无限

- 做 Context Budget Ledger：把 system policy、project memory、user goal、file reads、tool outputs、conversation 分桶展示。
- 稳定高频前缀：系统约束、项目规则、工具 schema 固定为可缓存前缀；一次性证据和大输出放后缀。
- 大输出先摘要后展开：日志、diff、网页先存原文引用，再送摘要和索引。
- 压缩前生成“将丢弃清单”：被剪掉的路径规则、文件证据、用户限制显性化，允许用户 pin。
- 把成本变成产品策略：低价值闲聊不占长任务会话；任务切换默认 /clear 或另开 episode。

4.3 Memory 分级：避免长期记忆污染任务执行

层级	内容	写入者	生命周期	风险
L0 Session Scratch	当前步骤观察、工具输出摘要	Harness	任务内	过期污染下一任务
L1 Task Memory	计划、状态差、关键证据、恢复包	Harness + 用户确认	完成后归档	错误计划被复用
L2 Project Memory	项目结构、构建命令、代码规范、固定工具	用户 / 团队	项目级	旧规则不更新
L3 User Preference	确认习惯、风险偏好、输出格式	用户 + Harness	用户级	偏好越权
L4 Training Candidate	脱敏后的成功 / 失败轨迹	数据管道	训练 / 评估集	隐私和标签噪声

硬规则： - 长期记忆必须有来源、最后验证时间、作用域、删除入口。 - 任何影响工具调用和权限的记忆必须可展示、可审计、可撤销。

5. 冷启动与内部 Dogfooding：首月拿下 3 个任务场景

冷启动不追全能。首月只拿边界清晰、状态差可验证、失败轨迹高价值、用户愿意反复使用的任务。

5.1 场景一：Issue -> Plan -> Diff -> Test -> PR

- 目标用户：内部研发、开源贡献者、重度 Agent 开发者。
- 边界：小 bug 修复、测试补全、依赖更新、文档代码片段修正。
- 工具链：Git、代码搜索、测试命令、lint、PR 生成、MCP issue tracker。
- 指标：计划采纳率、diff 接受率、测试通过率、人工接管率、回滚率、TRR。

5.2 场景二：本地资料 -> 结构化交付物 -> 人工确认 -> 发布

- 目标用户：非技术用户、产品、运营、研究支持角色。
- 边界：会议纪要、候选人材料归纳、社群反馈周报、竞品信息卡。
- 工具链：本地文件、浏览器、文档编辑、表格输出、消息草稿。
- 指标：首次有效任务时间、模板采纳率、引用完整率、人工修改比例、二次启动率。

5.3 场景三：工具链失败诊断与恢复助手

- 目标用户：开发者、内部研发、深度 Agent 用户。
- 边界：构建失败、测试失败、MCP 连接失败、权限错误、上下文压缩失败。
- 工具链：终端日志、配置文件、MCP health check、版本检查、重试与降级策略。
- 指标：失败恢复率、重复同错率、平均恢复时长、接管后成功率、恢复轨迹进 eval 率。

5.4 内部 Dogfooding：Harness Probe

组件	最小实现	输出
Local Event Collector	桌面端 / CLI sidecar 记录任务事件 JSONL；本地脱敏后上传	task_trace.jsonl
Plan Review Panel	展示计划、工具、风险、检查点；记录接受 / 修改 / 拒绝	plan_adoption_event
Tool Error Normalizer	把 shell、MCP、文件、网络错误映射到统一错误码	tool_failure_event
Takeover Recorder	接管时保存上下文、计划、最后工具结果、人工修复动作	takeover_bundle
ETCR Dashboard	按任务类型和用户层级展示漏斗	weekly_harness_report

5.5 首月 A/B 验证

阶段	实验	决策阈值
第 1 周	20-30 个内部真实任务跑 Probe，人工标注失败主因	字段完整率 $\geq 85\%$ ；主因标签一致率 $\geq 70\%$
第 2 周	计划确认 UI A/B：简版计划 vs 风险分层计划	风险分层计划让人工接管率下降，且计划采纳率不低于简版 90%
第 3 周	失败恢复 UI A/B：静默重试 vs 显示错误栈和备选路径	显示路径组重复同错率下降，任务中断率不上升
第 4 周	产出 Negative Trajectory Pack 和 Eval Regression Set	至少 50 条可复现失败轨迹；至少 20 条进入回归集

6. 极端场景防御：限流、缓存失效、工具链崩溃

桌面端的降级策略不能只写在后端告警里。用户必须知道现在还能做什么、不能做什么、什么时候必须阻断。

6.1 API 限流与高并发

- 分层队列：交互式确认 > 正在执行的状态改变 > 后台分析 > 批量总结。
- Plan-only 模式：额度不足时，只允许任务拆解、风险评估、人工执行指南，不执行工具调用。
- 本地摘要器降级：大日志和大文件先本地切块和摘要。
- 幂等任务键：重复提交合并，防止限流时任务风暴。
- 显性等待策略：显示排队位置、可取消、可转本地执行。

6.2 上下文缓存失效

- 关键状态改变前校验 context_manifest checksum，不一致时进入 Rehydrate。
- Rehydrate 只加载最小必要上下文：用户目标、最后确认计划、当前状态差、待执行工具、风险规则。
- 缓存失效后禁止高风险写操作，直到用户确认或 verifier 通过。
- 压缩失败或 thrashing 时停止重试，切换到分片读取、subagent 或人工接管包。

6.3 工具链崩溃

故障	产品级降级	阻断条件
MCP Server 断连	切换只读模式；展示最近健康状态；提供重新授权入口	涉及状态改变且无可用 verifier
Shell 命令超时	杀进程，保留 stdout/stderr，建议更小范围命令	命令可能删除 / 覆盖文件且无法回滚
文件写入失败	生成 patch 文件而不是直接写入	目标路径不在授权目录

故障	产品级降级	阻断条件
测试环境缺失	输出复现命令和缺失依赖；标记结果未验证	用户要求可部署结果但测试不可运行
工具返回超大输出	只保留原文引用和摘要索引；要求用户选择展开范围	大输出导致上下文反复填满

6.4 安全阻断原则

- 未知来源输入触发外部动作，默认阻断。
- 模型置信度低且动作不可逆，默认阻断。
- 上下文不一致且动作会改变本地文件、账号、资金、发布渠道，默认阻断。
- 用户未授权 MCP Server 资源访问，默认阻断。
- Agent 无法给出回滚路径，默认阻断。

7. 30 日执行面板：可交付物、实验、验收线

时间	交付物	关键动作	验收线
D1-D3	Harness Probe 原型	本地 JSONL 事件采集、计划确认面板、错误码映射	3 类任务全链路可记录；字段完整率 $\geq 80\%$
D4-D7	ETCR Dashboard	按用户层、任务类型、工具类型拆漏斗	能定位前三类失败主因；可导出周报
W2	MCP Capability Card	接入 3-5 个内部高频工具；展示权限和健康度	schema mismatch 可追踪；工具失败率可按 server 切分
W3	Context Ledger + Pruning UI	显性化上下文预算、剪枝、pin、checksum	长任务上下文修正率下降；重复解释次数下降
W4	Negative Trajectory Pack	输出失败轨迹、人工修复、verifier、训练候选标签	50 条可复现失败；20 条回归 eval；10 条 SFT 候选

关键取舍： - 不先做海量插件生态，先做高频工具的健康度、权限、错误恢复。 - 不先追全自动，先把计划确认、状态差验证、人工接管做成默认路径。 - 不先优化聊天留存，先优化 ETCR、TFR、TRR。 - 不把 Memory 当魔法，先做来源、作用域、删除入口、验证时间。 - 不让研究团队读原始产品吐槽，直接给结构化轨迹包、负样本、偏好对、回归集。

最终判断： - 首月必须证明用户愿意把真实任务交给 Agent。 - 首月必须证明失败可以恢复，而不是被掩盖。 - 首月必须证明失败轨迹能进入模型训练和 Harness 回归系统。

8. 资料依据与事实边界

DeepSeek 未公开内部进展不作判断。公开资料只覆盖竞品机制、MCP、KV Cache、eval 方法和桌面端约束。

- Anthropic Claude Code Overview: <https://code.claude.com/docs/en/overview>
- Anthropic Claude Code Context Window: <https://code.claude.com/docs/en/context-window>
- Anthropic Claude Code Permissions: <https://code.claude.com/docs/en/permissions>
- Anthropic Claude Code Troubleshooting: <https://code.claude.com/docs/en/troubleshooting>
- OpenAI Introducing Codex: <https://openai.com/index/introducing-codex/>
- OpenAI Codex CLI: <https://developers.openai.com/codex/cli>
- OpenAI Codex Cloud: <https://developers.openai.com/codex/cloud>
- OpenClaw README: <https://github.com/openclaw/openclaw>
- Model Context Protocol Specification: <https://modelcontextprotocol.io/specification/2025-06-18>
- Hugging Face Transformers Caching:
https://huggingface.co/docs/transformers/en/cache_explanation
- Hugging Face TGI PagedAttention: https://huggingface.co/docs/text-generation-inference/conceptual/paged_attention
- OpenAI Evals Guide: <https://developers.openai.com/api/docs/guides/evals>
- Agent-Diff: <https://arxiv.org/abs/2602.11224>